

PoC Task: End to End Pipeline for Hosting a simple website in local using Terraform and Jenkins

Overview

This documentation outlines the steps to create an end-to-end pipeline for hosting a static website on a local machine using an Nginx Docker image. The process includes setting up a Docker container for Nginx, automating deployment with Jenkins, inspection of code using SonarQube and provisioning infrastructure with Terraform. The Nginx Docker container serves as the local web server, providing a lightweight and easily configurable environment for hosting the website.

Prerequisites

Before starting, ensure the following tools are installed and configured on your local machine:

- Terraform

Oh, that's it. Terraform will take care the rest. I have used Linux operating system for this project using windows subsystem for Linux.

WSL

Windows Subsystem for Linux (WSL) lets developers run a GNU/Linux environment -- including most command-line tools, utilities, and applications -- directly on Windows, unmodified, without the overhead of a traditional virtual machine or dual-boot setup. WSL is like using Linux terminal in windows. Refer this link to install and use Linux in your machine [Install WSL | Microsoft Learn](#)

Terraform Installation

After successful installation of Linux in your machine, install terraform with the following steps.

1. Go to Terraform official website -> [Install | Terraform | HashiCorp Developer](#)
2. Scroll to see Linux operating system, select you Linux Distro and copy the commands or download binary files based on your machine

Linux

Package manager

Ubuntu/Debian CentOS/RHEL Fedora Amazon Linux Homebrew

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg  
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list  
sudo apt update && sudo apt install terraform
```

Binary download

386 Version: 1.9.4 Download ⬇	AMD64 Version: 1.9.4 Download ⬇
ARM Version: 1.9.4 Download ⬇	ARM64 Version: 1.9.4 Download ⬇

3. Paste the commands on the Linux terminal and press Enter to install terraform

```
ubuntu@SivaneshDsp:~$ wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg  
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list  
sudo apt update && sudo apt install terraform
```

4. To verify the successfully installation of Terraform, enter this command:

\$ terraform -version

```
ubuntu@SivaneshDsp:~$ terraform --version  
Terraform v1.9.4  
on linux_amd64
```

Step By Step Guide:

Step 1: Create a “main.tf” file

The Terraform setup is designed to automate the installation of Docker, run SonarQube docker image, download Jenkins and configure a Jenkins job for a deployment pipeline. This setup ensures that your environment is prepared with all necessary components, enabling seamless deployment and integration within your pipeline.

Create a main.tf file in the following directory and paste the given lines of code

Jenkins-setup-module/main.tf

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.1"
    }
    jenkins = {
      source = "overmike/jenkins"
      version = "0.6.1"
    }
  }
}

# Install Docker using the provided shell script
resource "null_resource" "install_docker" {
  provisioner "local-exec" {
    command = "chmod 777 ${path.module}/install-docker.sh"
  }
  provisioner "local-exec" {
    command = "bash -x ${path.module}/install-docker.sh"
  }
}

# Download Jenkins
resource "null_resource" "download_jenkins" {
  depends_on = [null_resource.install_docker]
  provisioner "local-exec" {
    command = "chmod 777 ${path.module}/install-jenkins.sh"
  }
  provisioner "local-exec" {
    command = "bash -x ${path.module}/install-jenkins.sh"
  }
}
```

```

}
# sonarqube docker image
resource "null_resource" "sonarqube-image" {
  depends_on = [null_resource.install_docker]
  provisioner "local-exec" {
    command = <<-EOF
    sudo docker volume create sonarqube_data
    sudo docker volume create sonarqube_logs
    sudo docker volume create sonarqube_extensions
    sudo docker run -d --name sonarqube \
      -p 9000:9000 \
      -v sonarqube_data:/opt/sonarqube/data \
      -v sonarqube_logs:/opt/sonarqube/logs \
      -v sonarqube_extensions:/opt/sonarqube/extensions \
      sonarqube:lts-community
    EOF
  }
}

```

This Terraform script is responsible for automating the installation of Docker, SonarQube and Jenkins on your local machine using shell scripts. It leverages the `null_resource` resource type to execute local commands, allowing the shell scripts to be run during the Terraform provisioning process.

Providers Configuration

```

...

terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.1"
    }
    jenkins = {
      source = "overmike/jenkins"
      version = "0.6.1"
    }
  }
}

```

...

- **local:** The local provider is included to enable local execution of scripts and commands as part of the Terraform provisioning process.
- **jenkins:** The jenkins provider is specified, which might be used in other parts of the project to interact with Jenkins resources. However, in this script, the jenkins provider is not actively used.

Install Docker using Shell Script

...

```
resource "null_resource" "install_docker" {
  provisioner "local-exec" {
    command = "chmod 777 ${path.module}/install-docker.sh"
  }
  provisioner "local-exec" {
    command = "bash -x ${path.module}/install-docker.sh"
  }
}
```

1. **Resource `null\_resource.install\_docker`:** This resource is responsible for executing the shell script `install-docker.sh` that installs Docker on your local machine.
2. **Provisioners:**
 - The first **local-exec** provisioner changes the file permissions of **install-docker.sh** to ensure it is executable.
 - The second **local-exec** provisioner runs the script using **bash -x**, which executes the script and prints each command being executed to the console, useful for debugging.

Install Jenkins using Shell Script

...

```
resource "null_resource" "download_jenkins" {
  depends_on = [null_resource.install_docker]
  provisioner "local-exec" {
    command = "chmod 777 ${path.module}/install-jenkins.sh"
  }
}
```

```

        provisioner "local-exec" {
            command = "bash -x ${path.module}/install-jenkins.sh"
        }
    }
}

```

1. **Resource null_resource.download_jenkins:** This resource handles the download and installation of Jenkins on your local machine.
2. **Depends_on:** This ensures that the download_jenkins resource will only run after the install_docker resource has successfully completed. This dependency ensures Docker is installed before Jenkins, as Jenkins requires Docker to run in this setup.
3. **Provisioners:**
 - The first **local-exec** provisioner changes the file permissions of **install-jenkins.sh** to make it executable.
 - The second **local-exec** provisioner runs **the install-jenkins.sh** script to download and install Jenkins.

Running Sonarqube using docker image

```

...

resource "null_resource" "sonarqube-image" {
    depends_on = [null_resource.install_docker]
    provisioner "local-exec" {
        command = <<-EOF
            sudo docker volume create sonarqube_data
            sudo docker volume create sonarqube_logs
            sudo docker volume create sonarqube_extensions
            sudo docker run -d --name sonarqube \
                -p 9000:9000 \
                -v sonarqube_data:/opt/sonarqube/data \
                -v sonarqube_logs:/opt/sonarqube/logs \
                -v sonarqube_extensions:/opt/sonarqube/extensions \
                sonarqube:its-community
        EOF
    }
}

```

EOF

```
}  
}  
...
```

1. Commands:

- **Create Volumes:** Creates three Docker volumes for SonarQube data, logs, and extensions.
- **Run SonarQube Container:** Runs the SonarQube container in detached mode (-d), exposing port 9000 and mounting the volumes.

2. Purpose:

- Creates three Docker volumes named sonarqube_data, sonarqube_logs, and sonarqube_extensions.

3. Use:

- These volumes are used to persist SonarQube data, logs, and extensions outside the container, ensuring data isn't lost when the container stops or is removed.

Step 2: Write a Shell Script to Install Docker and Jenkins

Create a install-docker.sh file in the same directory and write the following lines of code

[jenkins-setup-module/install-docker.sh](#)

```
#!/bin/bash  
  
# Add Docker's official GPG key:  
sudo apt-get update  
sudo apt-get install ca-certificates curl  
sudo install -m 0755 -d /etc/apt/keyrings  
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o  
/etc/apt/keyrings/docker.asc  
sudo chmod a+r /etc/apt/keyrings/docker.asc  
  
# Add the repository to Apt sources:  
echo \  
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]  
https://download.docker.com/linux/ubuntu \  
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \  

```

```
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update

#To install latest version
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin

# Add the current user to the Docker group
sudo usermod -aG docker $USER

# Print Docker version to confirm installation
sudo docker --version
```

The install-docker.sh script automates the installation of Docker on an Ubuntu system by:

1. Updating the system and installing required packages.
2. Adding Docker's official GPG key and repository.
3. Installing Docker from the official repository.
4. Adding the current user to the Docker group to enable Docker command execution without sudo.
5. Verifying the installation by printing the Docker version.

This script is an essential part of automating infrastructure setup, especially in environments where Docker is required for containerization tasks.

jenkins-setup-module/install-jenkins.sh

```
#!/bin/bash

# Update package index
sudo apt-get update

# Install Java (Jenkins requires Java)
sudo apt-get install -y openjdk-17-jre

# Verify java
java -version

# Adding Jenkins repository key...
curl -fsSL https://pkg.jenkins.io/debian/jenkins.io-2023.key | sudo tee
/usr/share/keyrings/jenkins-keyring.asc > /dev/null

# Add Jenkins repository
```

```
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]
https://pkg.jenkins.io/debian binary/ | sudo tee
/etc/apt/sources.list.d/jenkins.list > /dev/null

# Update package index again
sudo apt-get update

# Install Jenkins
sudo apt-get install -y jenkins

# Start Jenkins service
sudo systemctl start jenkins

# Enable Jenkins to start on boot
sudo systemctl enable jenkins

# permission to use docker
sudo usermod -aG docker jenkins

# Restart Jenkins
sudo systemctl restart jenkins

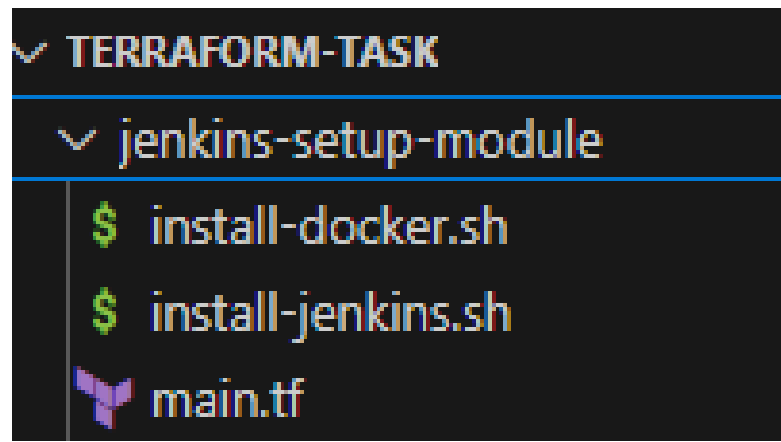
# Password for jenkins
echo "Jenkins is installed. The initial admin password is:"
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

The install-jenkins.sh script automates the installation of Jenkins on an Ubuntu system by:

1. Updating the package index and installing Java (required by Jenkins).
2. Adding the official Jenkins repository and its GPG key.
3. Installing Jenkins and configuring it to start as a service and to start automatically on boot.
4. Adding Jenkins to the Docker group to grant necessary permissions for Docker operations.
5. Retrieving and displaying the initial admin password for Jenkins setup.

This script is designed to streamline the setup of Jenkins, making it ready for use in CI/CD pipelines with Docker integration.

Till now your project structure will be...



Step 3: Create another main.tf file for building Jenkins pipeline

terraform-task/main.tf

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.1"
    }
    jenkins = {
      source = "overmike/jenkins"
      version = "0.6.1"
    }
  }
}

module "jenkins_setup" {
  source = "./jenkins-setup-module"
}
```

This Terraform script is responsible for automating the building and deploying process in Jenkins on your local machine using xml file. It leverages the `null_resource` resource type to execute local commands, allowing the xml file to be run during the Terraform provisioning process.

Jenkinsfile (in the repository, need not to create in local)

```
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = 'custom_nginx_image'
        CONTAINER_NAME = 'nginx_html'
        SONARQUBE_SERVER = 'SonarQube' // Replace with your SonarQube server name
    }

    in Jenkins
        PROJECT_KEY = 'poc' // Replace with your project key configured in
        SonarQube
            SONAR_SCANNER_HOME = tool name: 'SonarQube Scanner', type:
            'hudson.plugins.sonar.SonarRunnerInstallation'
        }

    stages {
        stage('Checkout') {
            steps {
                git url: 'https://github.com/sivanesh-dsp/Speed-Test.git',
                branch: 'main'
            }
        }

        stage('SonarQube Analysis') {
            steps {
                script {
                    // Run SonarQube analysis
                    withSonarQubeEnv(SONARQUBE_SERVER) {
                        sh "${SONAR_SCANNER_HOME}/bin/sonar-scanner -
                        Dsonar.projectKey=${PROJECT_KEY} -Dsonar.sources=."
                    }
                }
            }
        }

        stage('Build Docker Image') {
            steps {
                script {
                    // Build the Docker image using the Dockerfile in the
                    repository
                    sh "docker build -t $DOCKER_IMAGE ."
                }
            }
        }
    }
}
```

```

    stage('Deploy') {
        steps {
            script {
                // Stop and remove any existing container with the same name
                sh "docker rm -f $CONTAINER_NAME || true"

                // Run the container with the built image
                sh "docker run -d -p 5000:80 --name $CONTAINER_NAME
$DOCKER_IMAGE"
            }
        }
    }

    stage('Post-Deployment Test') {
        steps {
            script {
                // Test if the Nginx container is running and serving the
content
                sh 'curl -I http://localhost:5000 | grep "200 OK"'
            }
        }
    }
}

post {
    success {
        echo 'Deployment succeeded!'
    }
    failure {
        echo 'Deployment failed!'
    }
}
}

```

Dockerfile (in the repository, need not to create in local)

```

FROM nginx:latest

COPY ./ /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

```

terraform-task/job.xml

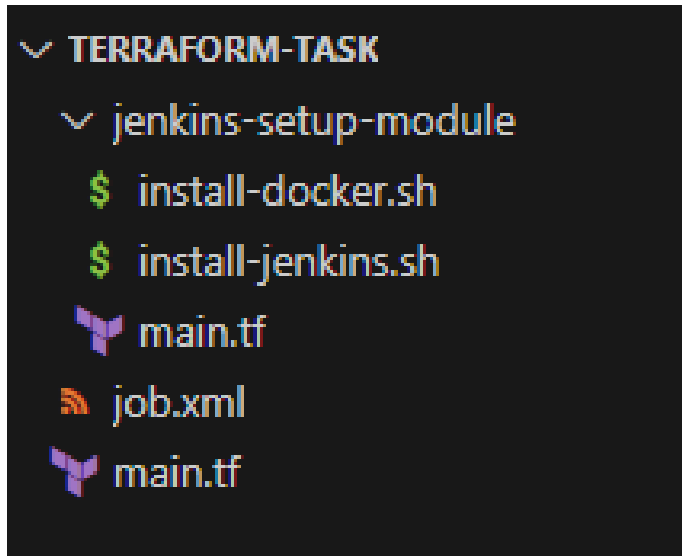
```
<?xml version='1.1' encoding='UTF-8'?>
<flow-definition plugin="workflow-job@2.25">
  <actions/>
  <description></description>
  <keepDependencies>false</keepDependencies>
  <properties/>
  <definition class="org.jenkinsci.plugins.workflow.cps.CpsScmFlowDefinition"
plugin="workflow-cps@2.59">
    <scm class="hudson.plugins.git.GitSCM" plugin="git@3.9.1">
      <configVersion>2</configVersion>
      <userRemoteConfigs>
        <hudson.plugins.git.UserRemoteConfig>
          <url>https://github.com/sivanesh-dsp/Speed-Test.git</url>
          <credentialsId></credentialsId>
        </hudson.plugins.git.UserRemoteConfig>
      </userRemoteConfigs>
      <branches>
        <hudson.plugins.git.BranchSpec>
          <name>*/main</name>
        </hudson.plugins.git.BranchSpec>
      </branches>
      <doGenerateSubmoduleConfigurations>false</doGenerateSubmoduleConfigur
ations>
      <submoduleCfg class="list"/>
      <extensions/>
    </scm>
    <scriptPath>Jenkinsfile</scriptPath>
    <lightweight>true</lightweight>
  </definition>
  <triggers>
    <hudson.triggers.SCMTrigger>
      <spec>* * * * *</spec>
    </hudson.triggers.SCMTrigger>
  </triggers>
  <disabled>false</disabled>
</flow-definition>
```

The job.xml allow the Jenkins to pull the repo of the simple website and find the Jenkinsfile in that repository. The Jenkinsfile is responsible for the pipeline that checkouts the github repo, inspects the code with the help of SonarQube, build the docker image and deploys into nginx container that has exposed (http://localhost:5000) port to access from your browser

Here, "<https://github.com/sivanesh-dsp/Speed-Test.git>" is the GitHub link for the repository change the link and branch name if required.

This job.xml is taken from the official Jenkins provider for terraform. Here is the link [jenkins_job](#)

Now, your project structure looks like this...



Step 4: terraform init

1. Open your Linux terminal and change directory to your project

```
ubuntu@SivaneshDsp:/mnt/c/Users/SIVANESH B/Desktop/terraform-task$ pwd
/mnt/c/Users/SIVANESH B/Desktop/terraform-task
ubuntu@SivaneshDsp:/mnt/c/Users/SIVANESH B/Desktop/terraform-task$
```

2. Enter `terraform init` command to perform several different initialization steps in order to prepare the current working directory for use with Terraform.

```
PowerShell
ubuntu@SivaneshDsp: /mnt/c/
ubuntu@SivaneshDsp: /mnt/c/Users/SIVANESH B/Desktop/terraform-task$ terraform init
Initializing the backend...
Initializing modules...
- jenkins_setup in jenkins-setup-module
Initializing provider plugins...
- Finding hashicorp/local versions matching "~> 2.1"...
- Finding overmike/jenkins versions matching "0.6.1"...
- Finding latest version of hashicorp/null...
- Installing overmike/jenkins v0.6.1...
- Installed overmike/jenkins v0.6.1 (self-signed, key ID EBB233FC42C79D77)
- Installing hashicorp/null v3.2.2...
- Installed hashicorp/null v3.2.2 (signed by HashiCorp)
- Installing hashicorp/local v2.5.1...
- Installed hashicorp/local v2.5.1 (signed by HashiCorp)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://www.terraform.io/docs/cli/plugins/signing.html
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
ubuntu@SivaneshDsp: /mnt/c/Users/SIVANESH B/Desktop/terraform-task$
```

3. Enter `terraform plan -target=module.jenkins_setup`` to create a execution plan, which lets you preview the changes that Terraform plans to make to your infrastructure.

```
PowerShell
ubuntu@SivaneshDsp: /mnt/c/
ubuntu@SivaneshDsp: /mnt/c/Users/SIVANESH B/Desktop/terraform-task$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# module.jenkins_setup.null_resource.download_jenkins will be created
+ resource "null_resource" "download_jenkins" {
  + id = (known after apply)
}

# module.jenkins_setup.null_resource.install_docker will be created
+ resource "null_resource" "install_docker" {
  + id = (known after apply)
}

Plan: 2 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run
"terraform apply" now.
ubuntu@SivaneshDsp: /mnt/c/Users/SIVANESH B/Desktop/terraform-task$
```

4. Enter `terraform apply -target=module.jenkins_setup`` to apply the changes required to reach the desired state of the configuration, or the pre-determined set of actions generated by a `terraform plan`` execution plan.

This steps downloads and installs the jenkins, setup of jenkins has to be done manually.

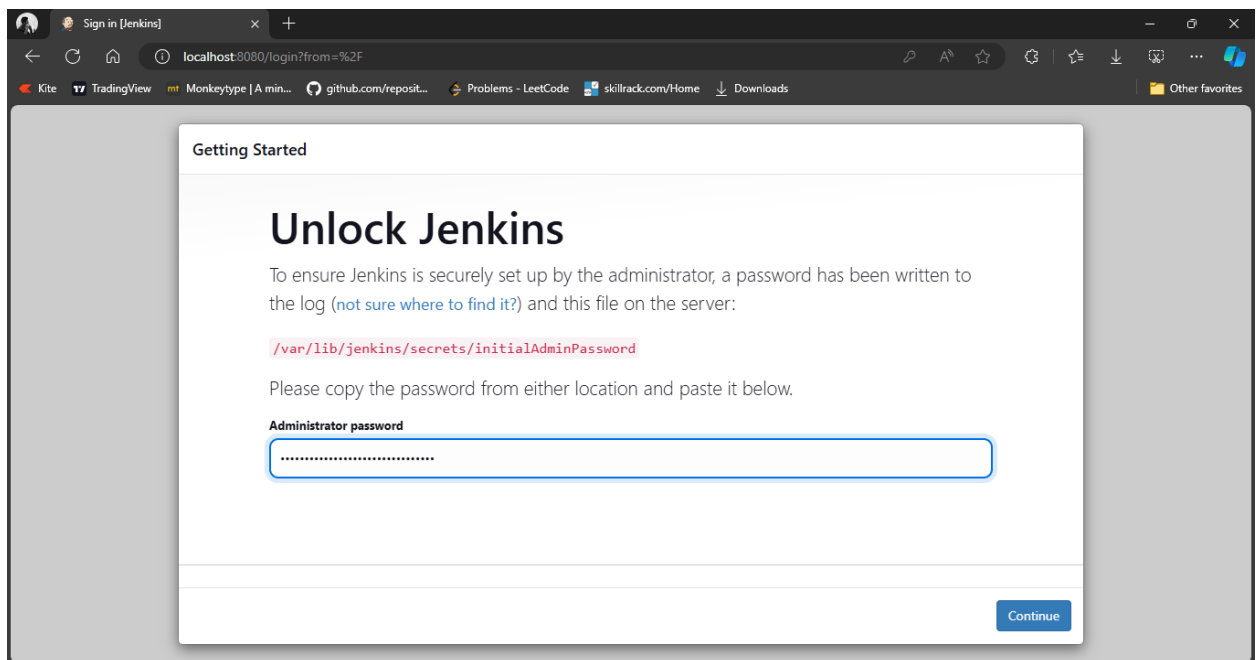
Step 5: Jenkins Setup

1. After completing the `terraform apply` command copy the string that shows in the output.

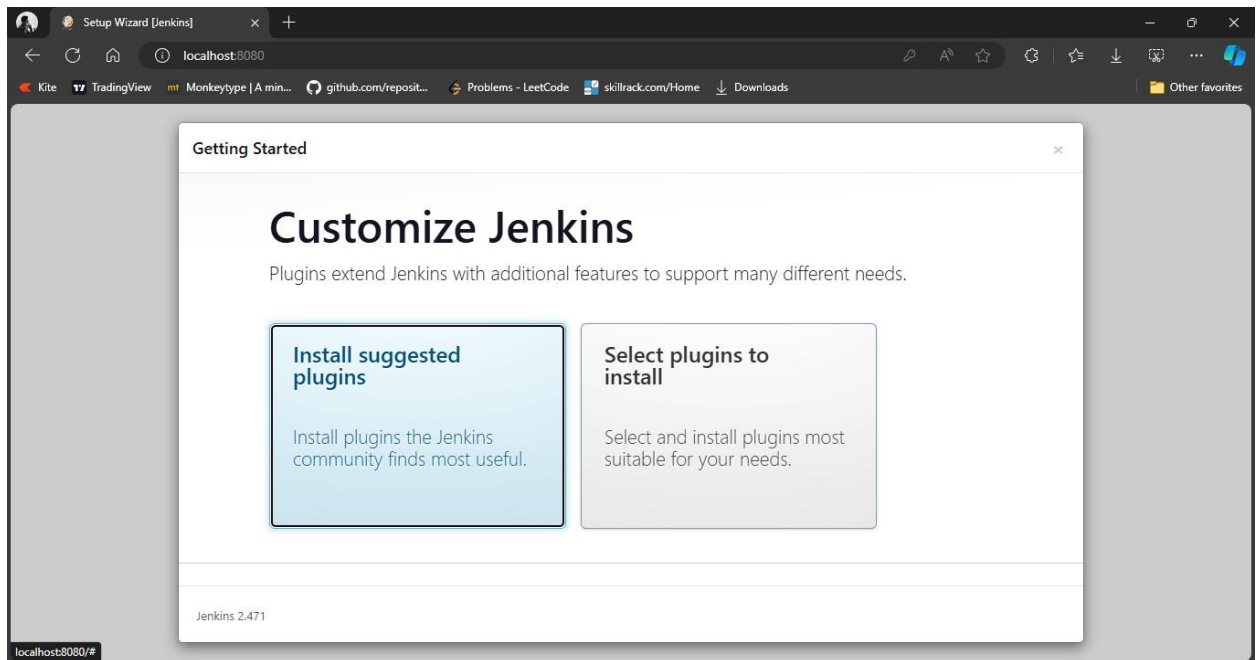
```
ubuntu@SivaneShDsp: ~$ terraform apply
module.jenkins_setup.null_resource.download_jenkins (local-exec): Preparing to unpack .../net-tools_1.60+git20181103.0eebece-1ubuntu5_
amd64.deb ...
module.jenkins_setup.null_resource.download_jenkins (local-exec): Unpacking net-tools (1.60+git20181103.0eebece-1ubuntu5) ...
module.jenkins_setup.null_resource.download_jenkins (local-exec): Selecting previously unselected package jenkins.
module.jenkins_setup.null_resource.download_jenkins (local-exec): Preparing to unpack .../archives/jenkins_2.471_all.deb ...
module.jenkins_setup.null_resource.download_jenkins (local-exec): Unpacking jenkins (2.471) ...
module.jenkins_setup.null_resource.download_jenkins (local-exec): Setting up net-tools (1.60+git20181103.0eebece-1ubuntu5) ...
module.jenkins_setup.null_resource.download_jenkins (local-exec): Setting up jenkins (2.471) ...
module.jenkins_setup.null_resource.download_jenkins (local-exec): Created symlink /etc/systemd/system/multi-user.target.wants/jenkins
.service → /lib/systemd/system/jenkins.service.
module.jenkins_setup.null_resource.download_jenkins: Still creating... [1m30s elapsed]
module.jenkins_setup.null_resource.download_jenkins: Still creating... [1m40s elapsed]
module.jenkins_setup.null_resource.download_jenkins (local-exec): Processing triggers for man-db (2.10.2-1) ...
module.jenkins_setup.null_resource.download_jenkins (local-exec): + echo 'Starting Jenkins...'
module.jenkins_setup.null_resource.download_jenkins (local-exec): Starting Jenkins...
module.jenkins_setup.null_resource.download_jenkins (local-exec): + sudo systemctl start jenkins
module.jenkins_setup.null_resource.download_jenkins (local-exec): + echo 'Enabling Jenkins to start on boot...'
module.jenkins_setup.null_resource.download_jenkins (local-exec): Enabling Jenkins to start on boot...
module.jenkins_setup.null_resource.download_jenkins (local-exec): + sudo systemctl enable jenkins
module.jenkins_setup.null_resource.download_jenkins (local-exec): Synchronizing state of jenkins.service with SysV service script wit
h /lib/systemd/systemd-sysv-install.
module.jenkins_setup.null_resource.download_jenkins (local-exec): Executing: /lib/systemd/systemd-sysv-install enable jenkins
module.jenkins_setup.null_resource.download_jenkins (local-exec): + sudo usermod -aG docker jenkins
module.jenkins_setup.null_resource.download_jenkins (local-exec): + sudo systemctl restart jenkins
module.jenkins_setup.null_resource.download_jenkins: Still creating... [1m50s elapsed]
module.jenkins_setup.null_resource.download_jenkins (local-exec): + echo 'Jenkins is installed. The initial admin password is:'
module.jenkins_setup.null_resource.download_jenkins (local-exec): Jenkins is installed. The initial admin password is:
module.jenkins_setup.null_resource.download_jenkins (local-exec): + sudo cat /var/lib/jenkins/secrets/initialAdminPassword
module.jenkins_setup.null_resource.download_jenkins (local-exec): 9e7420267181459daa2dc8c9b90a7283
module.jenkins_setup.null_resource.download_jenkins: Creation complete after 1m57s [id=1050630194907446917]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
ubuntu@SivaneShDsp: ~/terraform-task$
```

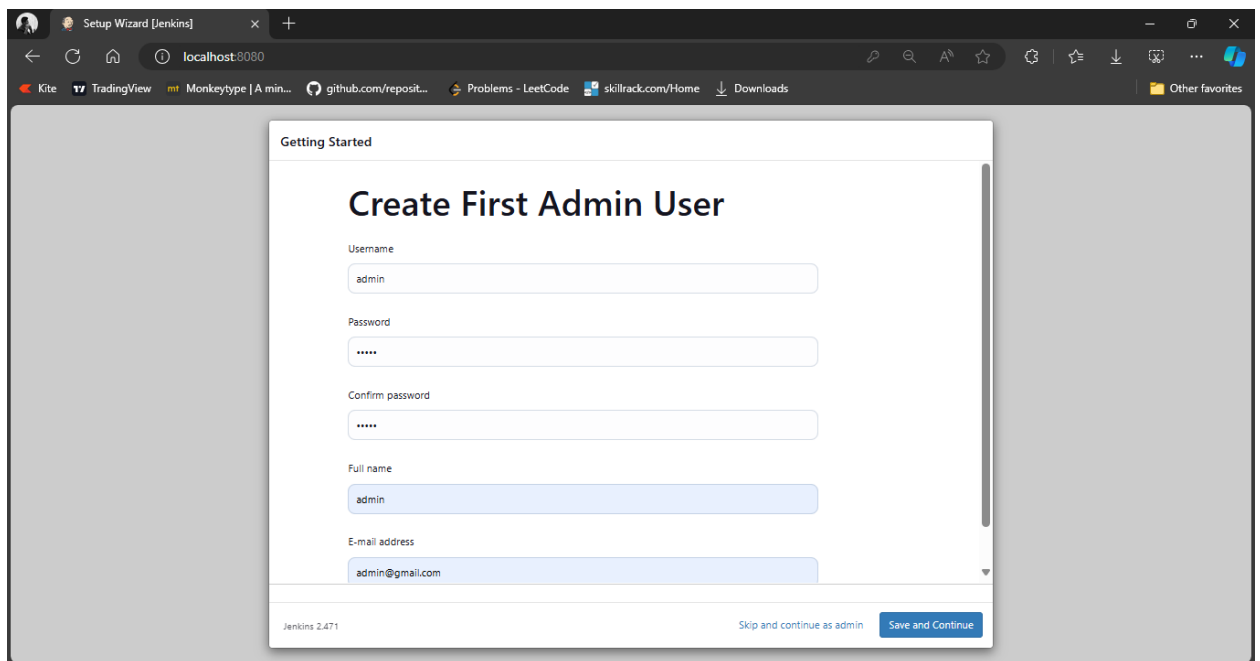
2. Visit <http://localhost:8080> and paste the string in the Administrator password field



3. Select "Install Suggested plugins" and wait for the installation of suggested plugins



4. Create username and password as “admin”

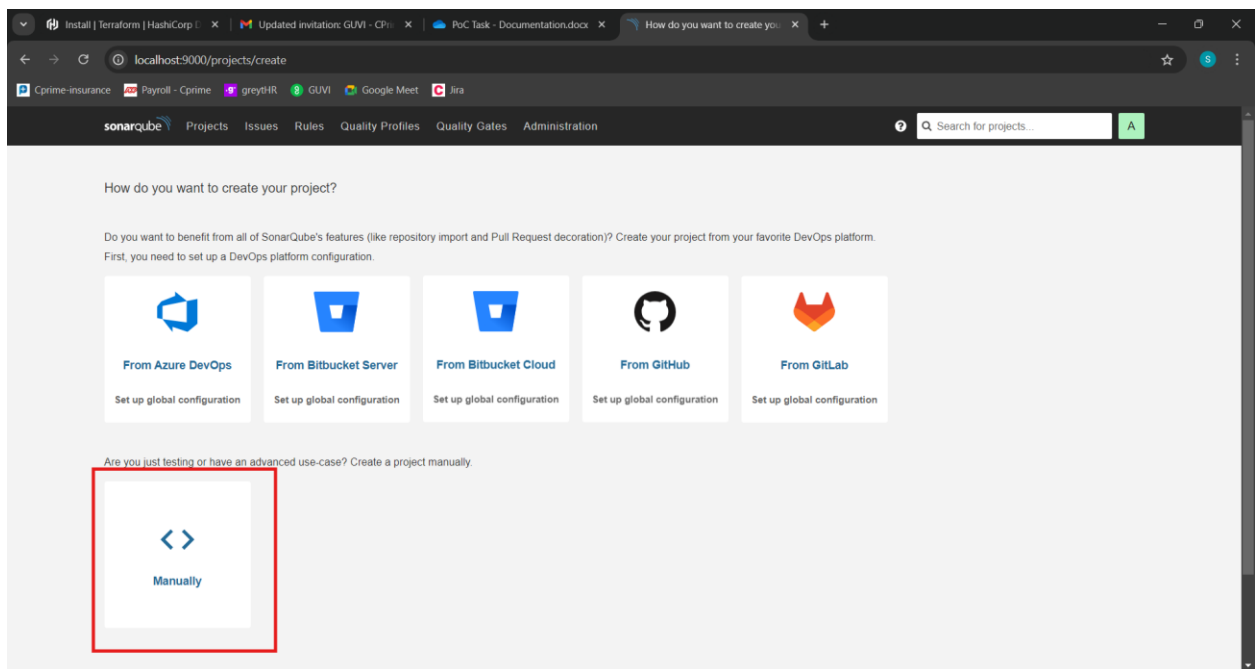


5. Select “Save and finish” in the next page
6. Click on “Start using jenkins” to view the Dashboard page in jenkins
7. Click on Manage Jenkins and then Plugins.
8. In the Available Plugins section, search for “Sonarqube Scanner” and install it.
9. Again, go to Manage Jenkins and then Tools.

10. In the Tools page come to “SonarQube Scanner installations” section and click on “Add Sonarqube Scanner”
11. Give a name for the scanner and Save

Step 6: Sonarqube setup

1. Visit <http://localhost:9000> on your browser to see sonarqube starting up.
2. Login into sonarqube with "admin" as username and "admin" as password (First time only)
3. Set New password for your admin account
4. After the setup, select Manual project in the dashboard



5. Enter Project Name as “poc” and press Set Up

localhost:5000/projects/create?mode=manual

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration

Create a project

All fields marked with * are required

Project display name *

poc

Up to 255 characters. Some scanners might override the value you provide.

Project key *

poc

The project key is a unique identifier for your project. It may contain up to 400 characters. Allowed characters are alphanumeric, '-' (dash), '_' (underscore), '.' (period) and ':' (colon), with at least one non-digit.

Main branch name *

main

The name of your project's default branch [Learn More](#)

[Set Up](#)

6. Now go to Administration and select Security -> Users.

localhost:5000/admin/users

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration

Administration

Configuration **Security** Projects System Marketplace

Users

Create and administer individual users. [Create User](#)

Search by login or name...

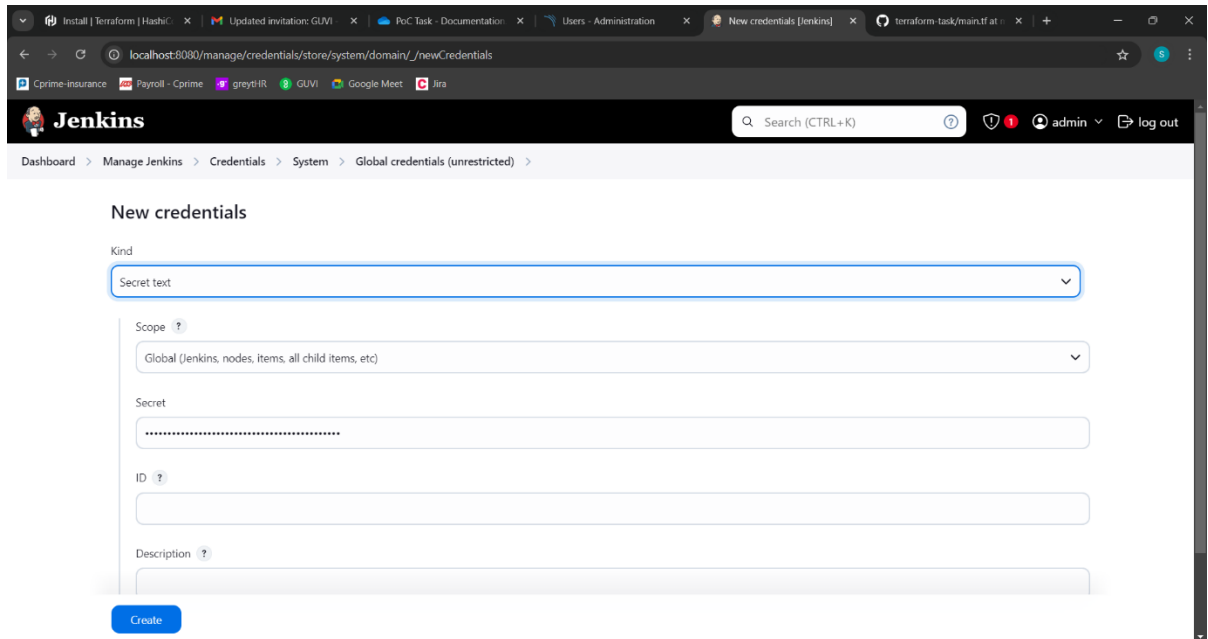
	SCM Accounts	Last connection	Groups	Tokens
A Administrator admin		< 1 hour ago	sonar-administrators sonar-users	0 Manage Settings

1 of 1 shown

Warning: Embedded database should be used for evaluation purposes only
The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.

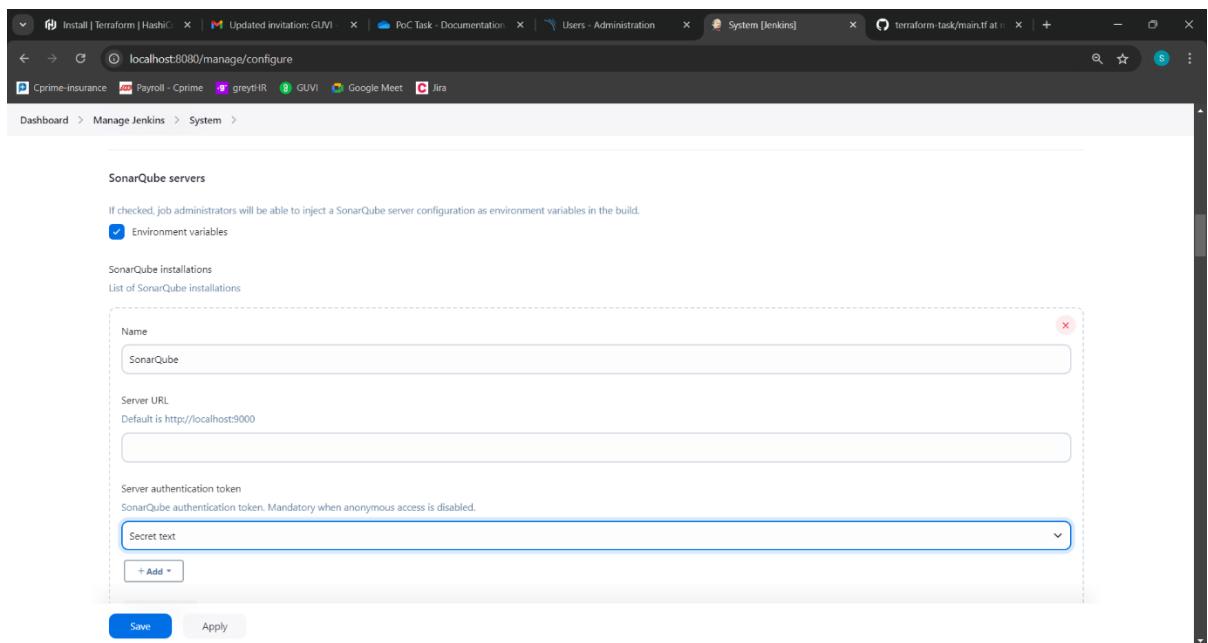
- Click on Tokens. Give a Name to that token and click generate.
- Copy that generated token for future use.
- Go to Jenkins (<http://localhost:8080>) Manage Jenkins -> Credentials.
- Click on Global and select "Add Credentials"

11. Select “Secret text” in the Kind and paste the generated token from the Sonarqube in the Secret field and click on Create.



The screenshot shows the Jenkins web interface at the URL `localhost:8080/manage/credentials/store/system/domain/_/newCredentials`. The breadcrumb trail is `Dashboard > Manage Jenkins > Credentials > System > Global credentials (unrestricted)`. The page title is "New credentials". The "Kind" dropdown is set to "Secret text". The "Scope" dropdown is set to "Global (Jenkins, nodes, items, all child items, etc)". The "Secret" field contains a masked token (dots). The "ID" and "Description" fields are empty. A blue "Create" button is at the bottom.

12. Again, Go to Manage Jenkins and then System.
13. In the SonarQube Servers sections, check the Environment Variables.
14. Give Name for SonarQube installations and Select Secret Text in the Server Authentication Token field.
15. Click on Apply and Save.



The screenshot shows the Jenkins web interface at the URL `localhost:8080/manage/configure`. The breadcrumb trail is `Dashboard > Manage Jenkins > System`. The page title is "SonarQube servers". A checkbox labeled "Environment variables" is checked. Below, the "SonarQube installations" section has a "List of SonarQube installations" table. The table has three rows: "Name" (SonarQube), "Server URL" (Default is `http://localhost:9000`), and "Server authentication token" (Secret text). A blue "Save" button and a grey "Apply" button are at the bottom.

Step 6: Jenkins Build and Deploy

After Successfully completing jenkins setup, Enter `terraform apply` again to automate the build and deploy process in jenkins.

```
ubuntu@SivaneshDsp: ~$ sudo systemctl enable jenkins.service
module.jenkins_setup.null_resource.download_jenkins: Still creating... [1m30s elapsed]
module.jenkins_setup.null_resource.download_jenkins: Still creating... [1m40s elapsed]
module.jenkins_setup.null_resource.download_jenkins (local-exec): Processing triggers for man-db (2.10.2-1) ...
module.jenkins_setup.null_resource.download_jenkins (local-exec): + echo 'Starting Jenkins...'
module.jenkins_setup.null_resource.download_jenkins (local-exec): Starting Jenkins...
module.jenkins_setup.null_resource.download_jenkins (local-exec): + sudo systemctl start jenkins
module.jenkins_setup.null_resource.download_jenkins (local-exec): + echo 'Enabling Jenkins to start on boot...'
module.jenkins_setup.null_resource.download_jenkins (local-exec): Enabling Jenkins to start on boot...
module.jenkins_setup.null_resource.download_jenkins (local-exec): + sudo systemctl enable jenkins
module.jenkins_setup.null_resource.download_jenkins (local-exec): Synchronizing state of jenkins.service with SysV service script with /lib/systemd/systemd-sysv-install.
module.jenkins_setup.null_resource.download_jenkins (local-exec): Executing: /lib/systemd/systemd-sysv-install enable jenkins
module.jenkins_setup.null_resource.download_jenkins (local-exec): + sudo usermod -aG docker jenkins
module.jenkins_setup.null_resource.download_jenkins (local-exec): + sudo systemctl restart jenkins
module.jenkins_setup.null_resource.download_jenkins: Still creating... [1m50s elapsed]
module.jenkins_setup.null_resource.download_jenkins (local-exec): + echo 'Jenkins is installed. The initial admin password is:'
module.jenkins_setup.null_resource.download_jenkins (local-exec): Jenkins is installed. The initial admin password is:
module.jenkins_setup.null_resource.download_jenkins (local-exec): + sudo cat /var/lib/jenkins/secrets/initialAdminPassword
module.jenkins_setup.null_resource.download_jenkins (local-exec): 9e7420267181459daa2dc8c9b90a7283
module.jenkins_setup.null_resource.download_jenkins: Creation complete after 1m57s [id=1050630194907446917]

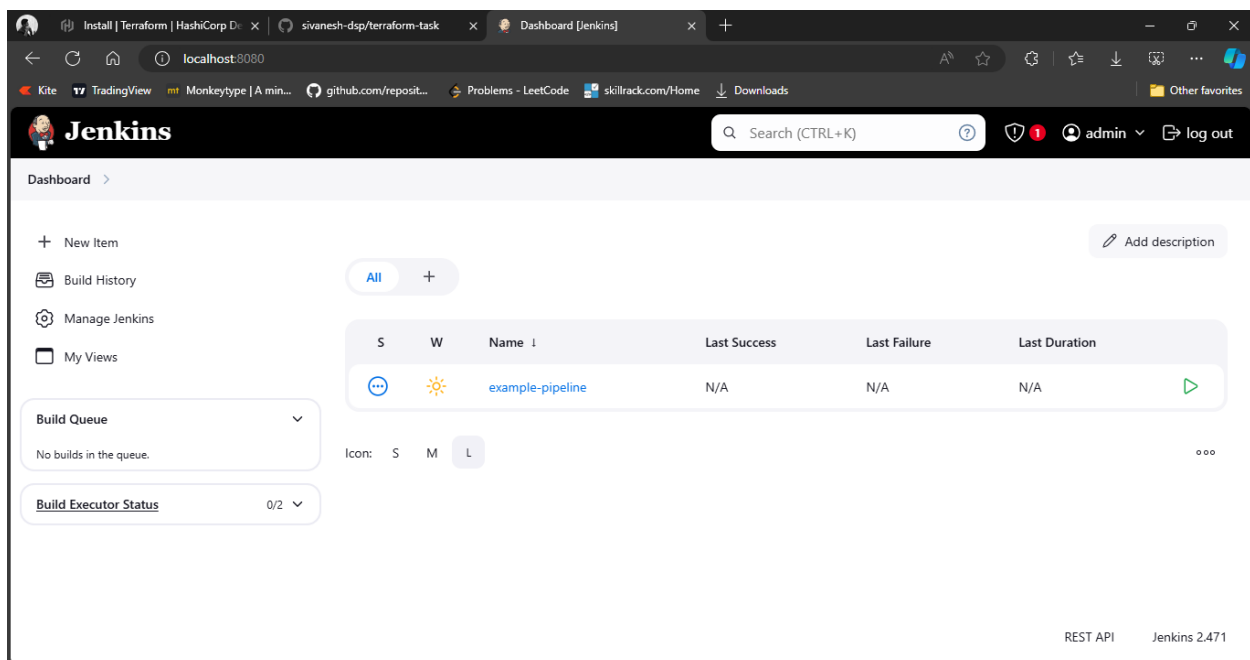
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
ubuntu@SivaneshDsp: ~/terraform-task$ terraform apply -auto-approve
module.jenkins_setup.null_resource.install_docker: Refreshing state... [id=706140449988001111]
module.jenkins_setup.null_resource.download_jenkins: Refreshing state... [id=1050630194907446917]

No changes. Your infrastructure matches the configuration.

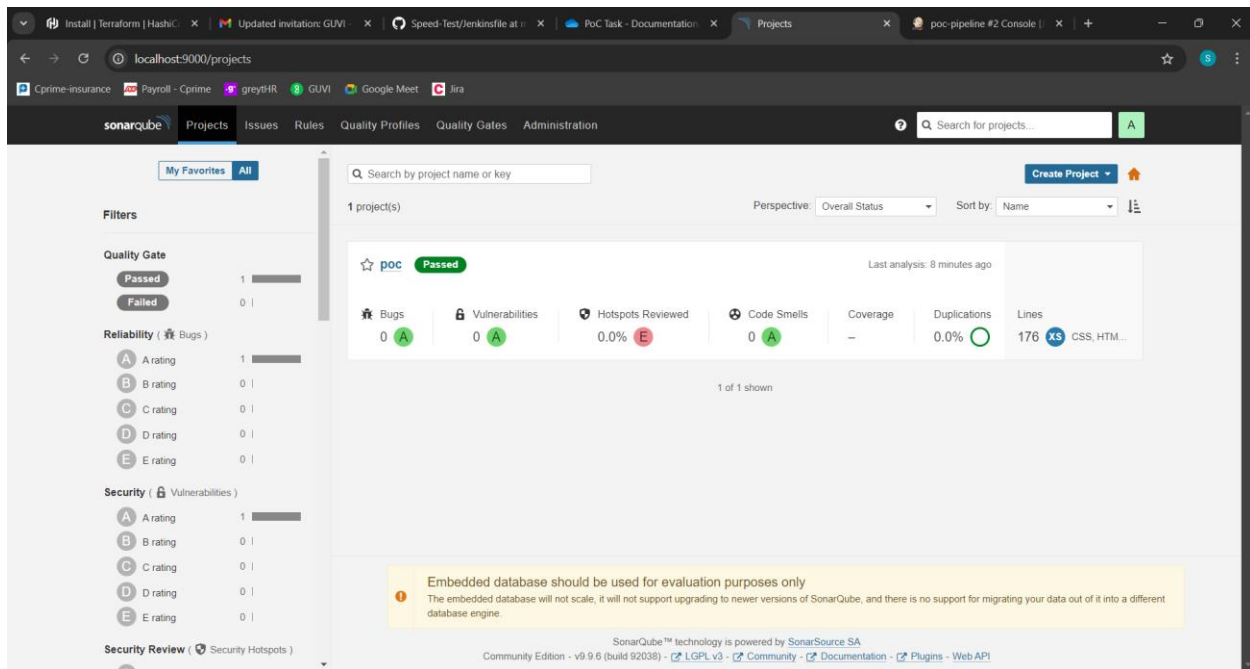
Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
ubuntu@SivaneshDsp: ~/terraform-task$
```

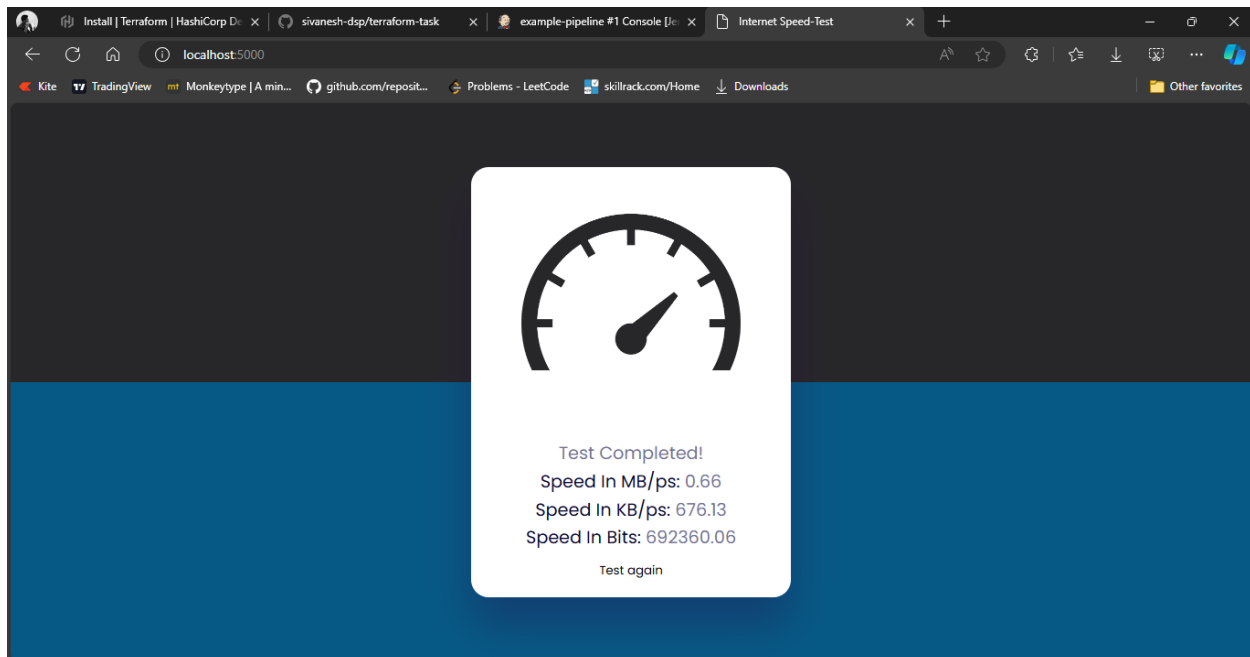
When you visit your localhost:8080, which is your Jenkins dashboard you can see the pipeline build automatically.



After the Successful Build, visit sonarqube project (<http://localhost:9000/projects>) you will see project report

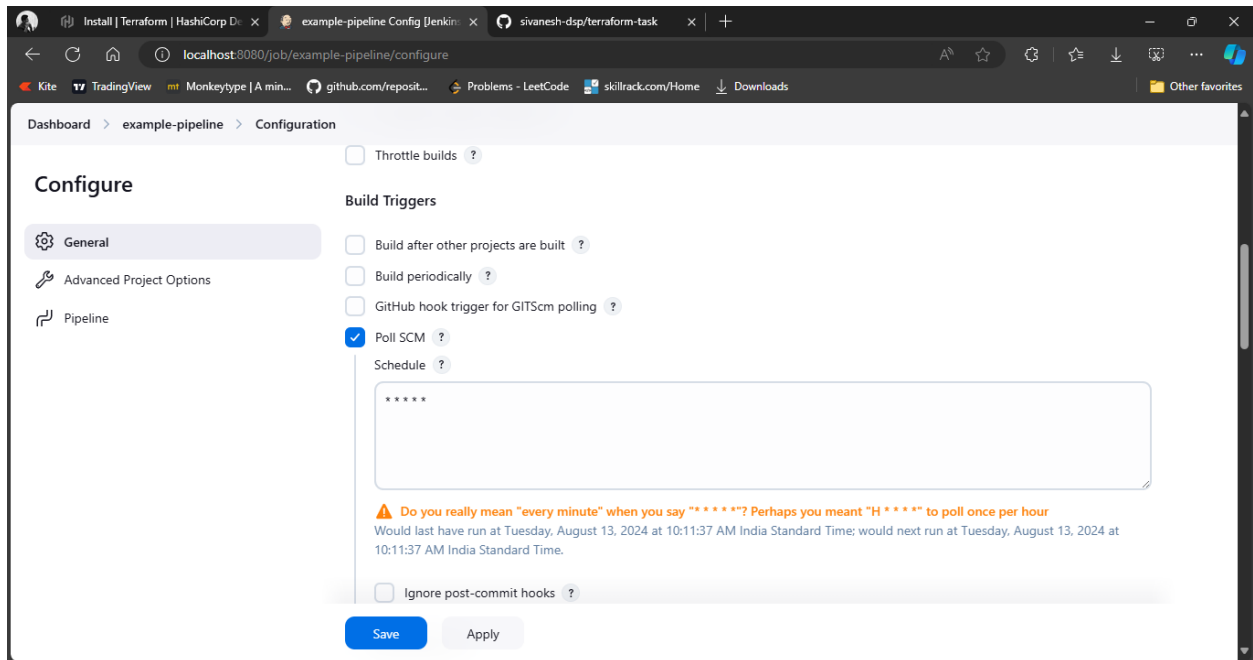


When you visit localhost:5000 you can see the website is deploy in your local machine.



Deployment completed...

In the configure section of the build, you can see “Build Triggers” option is set to “Poll SCM” automatically. Poll SCM options pulls the github code once every minute and only builds and deploys the code when is code is changed or modified.



Here is the sequence of commands.

1. terraform init
2. terraform plan -target=module.jenkins_setup
3. terraform apply -target=module.jenkins_setup -auto-approve
4. terraform apply -auto-approve

Conclusion

The website is successfully deployed locally, and Jenkins is configured to poll the SCM for updates, triggering builds and deployments automatically when changes are detected. This process efficiently sets up a local CI/CD pipeline that automates the deployment of a static website using Terraform and Jenkins, providing a robust solution for development and testing environments.